ЗВІТ

Практична робота №11

Тема: Агрегація, групування та операції над множинами

Мета роботи

Навчитися використовувати агрегатні функції SQL, виконувати групування даних за допомогою GROUP BY, фільтрувати групи за допомогою HAVING та застосовувати операції над множинами UNION, UNION ALL, INTERSECT і EXCEPT.

1. Підготовка бази даних

Для виконання практичної роботи використано раніше створену базу даних college.

Оскільки таблиця college.enrollments не містила стовпця для оцінок, було додано стовпець grade типу NUMERIC(5,2). Для демонстрації роботи агрегатних функцій були внесені тестові значення оцінок.

2. Використання агрегатних функцій та групування

Для аналізу даних використано такі агрегатні функції:

- **COUNT()** — підраховує кількість записів або значень.
- **AVG()** — обчислює середнє арифметичне значення.
- **MIN()** — знаходить мінімальне значення.
- **MAX()** — знаходить максимальне значення.

За допомогою GROUP BY було визначено кількість курсів на кожному факультеті. Також обчислено середню кількість кредитів за курсами та мінімальну й максимальну оцінки студентів.

Використання GROUP BY дозволяє об'єднувати записи за певною ознакою та виконувати агрегатні обчислення для кожної групи окремо.

3. Використання HAVING та порівняння з WHERE

За допомогою HAVING було відібрано курси, на які записано щонайменше двох студентів.

WHERE використовується для фільтрації окремих рядків до групування даних.

HAVING використовується для фільтрації вже сформованих груп після виконання GROUP BY.

Таким чином, WHERE працює з окремими записами, а HAVING — з результатами групування та агрегатними значеннями.

4. Операції над множинами

У практичній роботі використано чотири операції над множинами:

- **UNION** — об'єднує результати двох запитів та видаляє повторення.

- **UNION ALL** — об'єднує результати, зберігаючи всі повторення.
- **INTERSECT** — повертає спільні записи двох результатів.
- **EXCEPT** — повертає записи першого результату, яких немає у другому.

Для демонстрації операцій було використано ідентифікатори курсів факультетів IT та FIZ.

Кількість отриманих записів для кожної операції підраховується окремим запитом у файлі aggregates.sql. Оскільки ідентифікатори курсів належать лише одному факультету, у початковому наборі даних очікується:

Операція	**Кількість записів**
UNION	3
UNION ALL	3
INTERSECT	0
EXCEPT	2

5. Практичне завдання: топ курсів за кількістю студентів

Було створено запит для визначення десяти курсів із найбільшою кількістю записаних студентів.

Для цього використано:

- LEFT JOIN для об'єднання таблиць курсів та записів;
- COUNT() для підрахунку студентів;
- GROUP BY для групування за курсами;
- ORDER BY для сортування за кількістю студентів;
- LIMIT 10 для обмеження результату десятьма записами.

Такий запит дозволяє визначити найбільш популярні курси.

6. Контрольні запитання

1. Чому WHERE не може фільтрувати дані після групування?

WHERE виконується до GROUP BY та фільтрує окремі рядки. Для фільтрації сформованих груп використовується HAVING.

2. Що повертає COUNT(*) при відсутності рядків?

Якщо запит виконує COUNT(*) без GROUP BY і не знаходить жодного рядка, він повертає значення 0. Якщо використовується GROUP BY і рядків немає, результат не міститиме груп.

3. Чим відрізняються UNION та UNION ALL?

UNION об'єднує результати та видаляє дублікати. UNION ALL зберігає всі записи, включаючи повторення.

4. Як NULL впливає на агрегатні функції?

Функції AVG(), MIN() та MAX() ігнорують значення NULL. COUNT(стовпець) підраховує лише ненульові значення, а COUNT(*) враховує всі рядки.

Висновок

Під час виконання практичної роботи було опрацьовано агрегатні функції SQL, групування даних за допомогою GROUP BY та фільтрацію груп за допомогою HAVING. Також було розглянуто відмінності між WHERE та HAVING і практично застосовано операції UNION, UNION ALL, INTERSECT та EXCEPT.

Отримані навички дозволяють виконувати груповий аналіз даних, формувати статистичні звіти та об'єднувати результати SQL-запитів.